

Arkitekturë e thjeshtë për porositje B2B

Një monolit modular me PostgreSQL dhe API REST mban kompleksitetin operacional të ulët, por vendos kufij të qartë që mund të ndahen vetëm kur ngarkesa ose ekipet e justifikojnë.

Supozimi kryesor: një porosi ka produkte nga vetëm një merchant. Nëse operatori zgjedh merchant tjetër, krijohet draft i veçantë. Kjo thjeshton rezervimin e stokut, fulfillment-in, statusin dhe faturimin; `orders.merchant_id` është i detyrueshëm.

1. Organizimi i aplikacionit dhe modulet

Operator Web

Merchant Web

API / AuthZ

Catalog + Orders

PostgreSQL

- **Operator:** katalog i lexueshëm, kërkim/filtrim, draft dhe krijim porosie, histori dhe detaj.
- **Merchant:** CRUD i produkteve dhe stokut të vet; lexim i porosive që duhet të përmbushë.
- **Shared Catalog:** query, kategori, çmim dhe disponueshmëri; e njëjta logjikë domeni përdoret nga të dy UI-të.
- **Orders:** validim, snapshot, kontroll atomik stoku dhe status. **Auth:** identitet, sesion dhe membership.
- **Backend boundary:** UI nuk lidhet me DB. Handler-at thërrasin service-at e domenit; repository-t kontrollojnë PostgreSQL. Deploy i vetëm në MVP, jo microservices.

2. Rolet dhe lejet

Operator

Lexon katalogun e lejuar për organizatën e vet; krijon porosi vetëm për `operator_org_id` ku ka membership aktiv; lexon vetëm porositë e asaj organizate. Nuk ndryshon produkte ose stok.

Merchant

Lexon dhe ndryshon vetëm produktet ku `merchant.organization_id` përputhet me membership-in e vet; lexon vetëm porositë me `merchant_id` të tij. Nuk sheh porosi të merchant-ëve të tjerë.

Zbatim server-side: middleware verifikon sesionin; service-i nxjerr membership-et nga DB; çdo query përfshin `organization_id/merchant_id`. ID-të nga klienti nuk konsiderohen autorizim. Testet e integritetit provojnë aksesin cross-tenant dhe përgjigjen 403/404.

3. Entitetet dhe marrëdhëniet

Entiteti	Fushat / marrëdhënia kryesore
User	Identitet global; lidhet N:M me Organization përmes Membership.
Organization	Tenant/operator ose pronar merchant-i. Membership: <code>user_id</code> , <code>organization_id</code> , <code>role</code> .
Merchant	I përket një Organization; ka shumë Products dhe Orders.
Product	<code>merchant_id</code> , SKU unik brenda merchant-it, <code>name</code> , <code>category</code> , <code>price_minor</code> , <code>currency</code> , <code>stock</code> , <code>active</code> .
Order	<code>operator_org_id</code> , <code>merchant_id</code> , <code>status</code> , <code>currency</code> , <code>total_minor</code> , <code>created_by</code> , <code>timestamps</code> .
OrderItem	<code>order_id</code> , <code>product_id</code> , <code>quantity</code> ; ruan <i><code>product_name_snapshot</code></i> , <i><code>sku_snapshot</code></i> dhe <i><code>unit_price_minor_snapshot</code></i> .

Snapshot-et ruajnë historinë edhe kur emri ose çmimi i produktit ndryshon. Totali i porosisë llogaritet nga snapshot-et në të njëjtin transaksion; paraja ruhet në minor units me currency ISO.

4. Endpoint-et e nevojshme

Metoda dhe path	Roli	Rregulli bazë
GET /v1/catalog/products?q=&category=&merchantId=&cursor=	Operator	Vetëm produkte aktive/dukshme për organizatën; përgjigje e paginuar.
GET /v1/catalog/products/:id	Operator	I njëjti scope katalogu; 404 kur produkti nuk lejohet.
GET /v1/merchant/products	Merchant	Vetëm produktet e merchant-it të membership-it aktiv.
POST /v1/merchant/products	Merchant	Krijon produkt vetëm nën merchant-in e vet; validon SKU, currency, çmim dhe stok.
PATCH /v1/merchant/products/:id	Merchant	Ndryshon vetëm produkt në pronësi; optimistic version për konfliktet.
DELETE /v1/merchant/products/:id	Merchant	Soft-delete/deaktivizim në pronësi; nuk fshin historinë e porosive.
POST /v1/orders	Operator	Body: merchantId + [{productId, quantity}]. Serveri vendos operator_org_id dhe çmimet; kontroll atomik i stokut.
GET /v1/orders?cursor=&status=	Të dy	Operator: organizata e vet. Merchant: porositë e merchant-it të vet.
GET /v1/orders/:id	Të dy	Detaj vetëm kur order-i është brenda scope-it të rolit.

Nuk propozohen endpoint-e pagese, dërgese, analitike ose njoftimesh sepse nuk janë pjesë e rrjedhës së kërkuar.

5. Autentikimi dhe mbrojtja e të dhënave

- **Identiteti:** provider i provuar (p.sh. Auth0/Clerk/Keycloak ose framework auth i mirëmbajtur), OIDC dhe cookie sesioni HttpOnly, Secure, SameSite. Nëse mbahen passwords, provider-i menaxhon hashing, reset dhe MFA; aplikacioni nuk shpik kriptografi.
- **Autorizimi:** çdo request verifikon sesionin dhe membership-in në server. Rate limits, audit log për ndryshime merchant dhe secrets vetëm në secret manager. TLS në transit; enkriptim i storage/backups.
- **Validimi:** schema server-side për params/body; limite sasje, integer minor units, currency e njëjtë, UUID të vlefshme dhe madhësi të kufizuara. Mesazhet nuk zbulojnë tenant tjetër.
- **Krijimi i porosisë:** një transaksion DB kyç rreshtat e produkteve (SELECT ... FOR UPDATE ose update kushtor), rikontrollon active/merchant/stock, ul stokun, shkruan Order + snapshot-et dhe commit-on. Mungesa e stokut rollback-on gjithë porosinë me konflikt të qartë.

Ndarje e qartë: login-i i prototipit është vetëm state në browser dhe nuk është autentikim. Qasja e mësipërme është dizajni i prodhimit.

6. MVP kundrejt fazës së dytë

MVP — domosdoshme

- Login real, memberships dhe izolim tenant.
- Merchant CRUD për katalog dhe stok.
- Browse/search/filter për operatorin.
- Draft, validim dhe krijim transaksional porosie.
- Listë/detaj porosie për të dy rolet.
- Logging, monitoring gabimesh, backups dhe teste autorizimi.

Faza 2 — shtyhet

- Pagesa, kredi dhe faturim.
- Njoftime email/SMS dhe workflow i avancuar statusi.
- Multi-merchant checkout dhe ndarje automatike.
- Analitika, rekomandime dhe kërkim i avancuar.
- Rezervim stoku me timeout, import masiv dhe integritet ERP.

Arsyeja e shtyrjeve: pagesa, multi-merchant dhe integritet shtojnë pajtim financiar, rollback të shpërndarë dhe support operacional. MVP duhet së pari të provojë rrjedhën browse-to-order, pronësinë dhe saktësinë e stokut me sa më pak pika dështimi.

Evolucioni

Modulet publikojnë evente të brendshme pas commit-it. Kur volumi e kërkon, njoftimet ose analitika mund t'i konsumojnë asinkronisht përmes outbox pattern; Catalog dhe Orders ndahen në shërbime vetëm me metrika konkrete të ngarkesës ose pronësi ekipesh.